

Sec 10. 优化方法

- ① 代码块
- ② 基本块
- ③ 局部优化
- ④ 全局优化
- ⑤ 循环优化
- ⑥ 常量传播
- ⑦ 死代码消除
- ⑧ 控制流图
- ⑨ 数据流图
- ⑩ 图灵机

Sec 10. 优化

Overview

- 等价性计算
- 数据流图
- 局部优化
- 循环优化

代码块：强类型语言，删除基本/归纳变量

基本块

Terms:

$x := y + z$
对 x 赋值，引用 y, z
活跃：本基本块/其他基本块中被引用

划分：

主要看 goto，把 goto 的几块划分即可。
 { goto 到的块
 写 goto 的块与

局部优化

合并已知量

临时变量改名

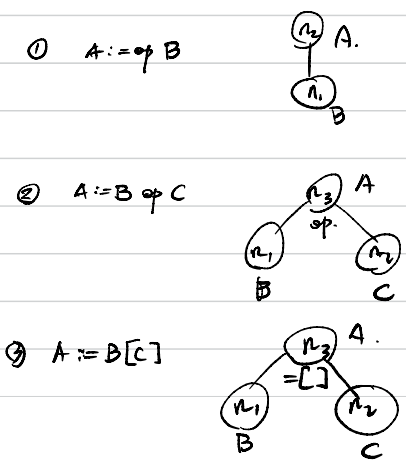
变换语句位置

代数变换：
 $y * 2 \Rightarrow y + y$
 $x := x + 0 \Rightarrow x := x * 1$

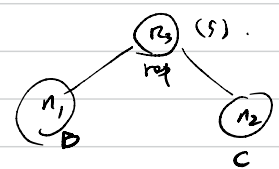
程序流图

上述基本块流程的图表示

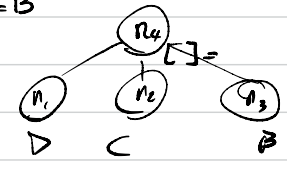
基本块的 DAG (另一种表示方式)



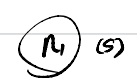
④ if B op C goto (S)



⑤ D[C] := B

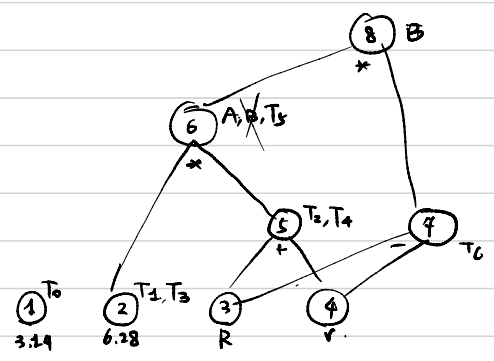


⑥ goto (S)



<e.g.>

- $T_0 := 3.14$
- $T_1 := 2 * T_0$
- $T_2 := R + r$
- $A := T_1 * T_2$
- $B := A$
- $T_3 := 2 * T_0$
- $T_4 := R + r$
- $T_5 := T_3 * T_4$
- $T_6 := R - r$
- $B := T_5 * T_6$



- 优化后：
- (1) $T_0 := 3.14$
 - (2) $T_1 := 6.28$
 - (3) $T_3 := 6.28$
 - (4) $T_2 := R + r$
 - (5) $T_4 := T_2$
 - (6) $A := 6.28 * T_2$
 - (7) $T_5 := A$
 - (8) $T_6 := R - r$
 - (9) $B := A * T_6$
- 若仅 A, B 活跃
 $T_2 = R + r$
 $A = 6.28 * T_2$
 $T_6 = R - r$
 $B = A * T_6$
 只计算相关变量即可

循环优化

代码块

循环不变：
 $const \ op \ const \rightarrow \ const$
 循环不改的变量 (视作 const)
 定值到达

外理保证不变运算是所有出口边
 已在循环中不再赋值了

• Algo. 找循环不变运算

① 找不变运算

② 对每个不变运算检查，(1) 或 (2) 成立。

(1) 必经、未定义值、用定值一统都是一个定值

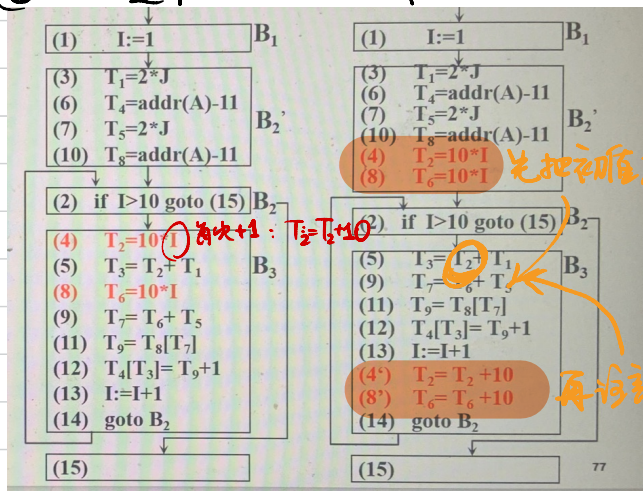
(2) A 在离开 L 后不是活跃的值 (1) 和 (2) 都

注不用了

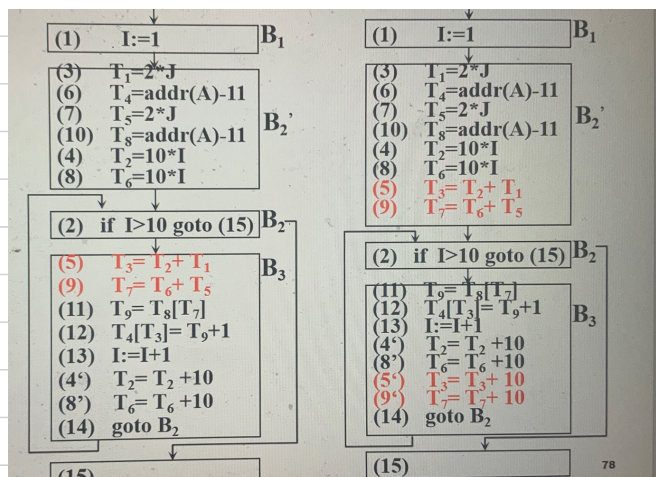
③ 提出去 (注意顺序)

• 强度削弱

① 运算: $\ast$ in loop $\rightarrow +$

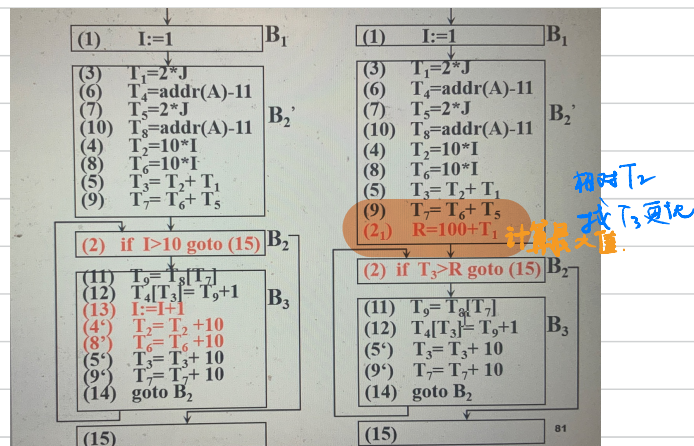


② 线性关系

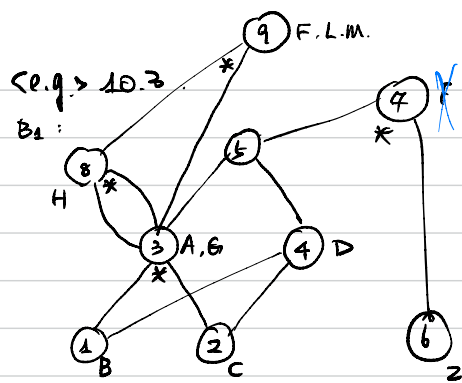


• 删除归纳变量

• 循环中只有 $i++$ 这种操作以操作为判断的，我把它线性相关变量，用那个判断可以少一个变量 i 。



归纳变量线性关系



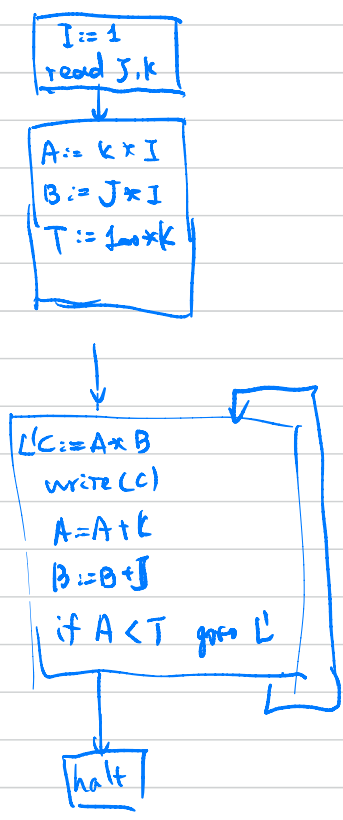
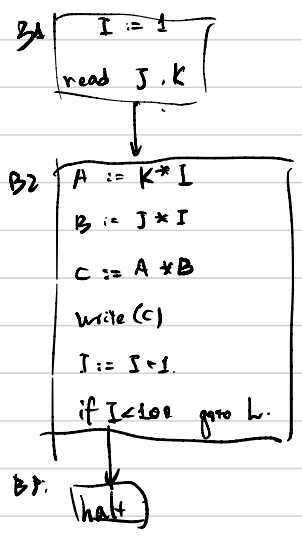
只 B, L, M 被引用

$G := B * C$
 $H := G * G$
 $L := H * G$
 $M := L$

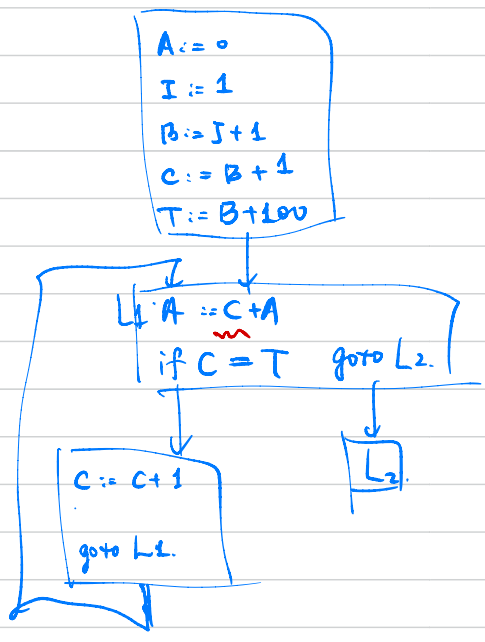
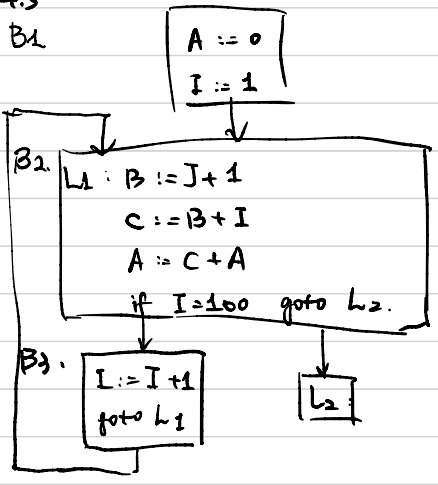
只 L 被引用

$G := B * C$
 $H := G * G$
 $L := G * H$

ce.g. > 4.5



ce.g. > 4.5



Sec 11. 目标代码生成

- ③ 优化寄存器分配: 待用活区信息表, 生成目标代码, 简单代码块

Sec 11. 目标代码生成

基本问题

- 指令选更快更好的
- 寄存器分配
- 计算顺序

目标机器模型

- 四种类型指令形式

类型	指令形式	意义(设 op 是二目运算符)
直接地址型	op R _i , M	(R _i) op (M) $\Rightarrow$ R _i
寄存器型	op R _i , R _j	(R _i) op (R _j) $\Rightarrow$ R _i
变址型	op R _i , c(R _j)	(R _i) op ((R _j)+c) $\Rightarrow$ R _i
间接型	op R _i , *M	(R _i) op ((M)) $\Rightarrow$ R _i
	op R _i , *R _j	(R _i) op ((R _j)) $\Rightarrow$ R _i
	op R _i , *(R _j)	(R _i) op (((R _j)+c)) $\Rightarrow$ R _i

指令

指令	意义
LD R _i , B	把 B 单元的内容取到寄存器 R _i , 即 (B) $\Rightarrow$ R _i
ST R _i , B	把寄存器 R _i 的内容存到 B 单元, 即 (R _i) $\Rightarrow$ B
J X	无条件转向 X 单元
CMP A, B	把 A 单元和 B 单元的值进行比较, 并根据比较情况把机器内部特征寄存器 CT 置成相应状态. CT 占两个二进位. 根据 A < B 或 A = B 或 A > B 分别置 CT 为 0 或 1 或 2.
J < X	如 CT=0 转 X 单元
J $\leq$ X	如 CT=0 或 CT=1 转 X 单元
J = X	如 CT=1 转 X 单元
J $\neq$ X	如 CT $\neq$ 1 转 X 单元
J > X	如 CT=2 转 X 单元
J $\geq$ X	如 CT=2 或 CT=1 转 X 单元

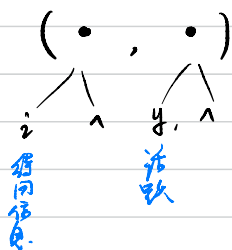
还有 ADD, SUB, DIV 等基本运算
MUL

简单代码生成

待用信息

i: A := B op C
j: D := A op E < 中间无其他定值了

待用活区表示



注: (x, x) $\rightarrow$ (x, x)
两边代码前边

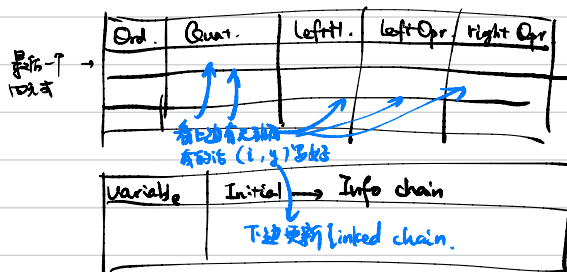
注: 默认所有说明变量在基本块
出口后都活区

待用活区表步骤

① 初始化: 出口活区变量: (^, y)

② 从后向前处理四元式

③ 对于 A := B op C



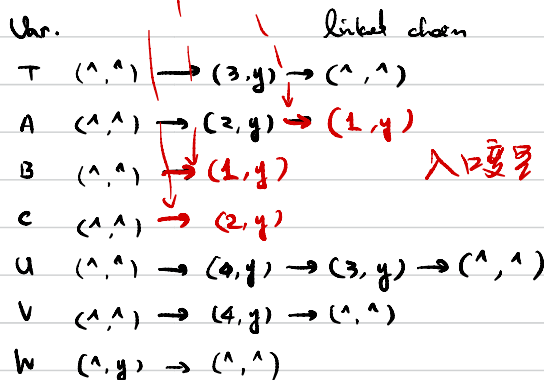
< e.g. > 1. T := A - B

2. U := A - C W active.

3. V := T + U

4. W := V + U

Ord.	Quant.	LH.	LOp.	ROp.
(4)	W := V + U	(^, y)	(^, ^)	(^, ^)
(3)	V := T + U	(4, y)	(^, ^)	(4, y)
(2)	U := A - C	(3, y)	(^, ^)	(1, ^)
(1)	T := A - B	(3, y)	(2, y)	(^, ^)



寄存器描述 & 地址描述

如 B := A 这种
RVALUE[R] = {A, B} 记录 Reg 寄存器

AVALUE[A] = {R1, R2, A} 记录 A 的活在哪里

代码生成算法.

对四元式 $i: A := B \text{ op } C$

① GETREG($i: A := B \text{ op } C$) 返回 R 存 A .

② 查 $AVALUE[B]$, $AVALUE[C] = B', C'$

③ 若 $B' \neq R$ 就是左操作数不在寄存器中.

生成 $LD\ R, B'$
 $op\ R, C'$

若 $B' = R$.

生成 $op\ R, C'$

④ Update $AVALUE[A] = \{R\}$, $RVALUE = \{A\}$.

⑤ 若 B, C 还有用不, 没用就把 $RVALUE, AVALUE$ 中
 对应部分删了.

• GETREG($i: A := B \text{ op } C$)

① 尽可能用 B 和 C Reg

② 尽可能用空闲 Reg.

③ 抢非空闲注意往后看这个[抢]引用情况的

• 抢了之后用不用 ST 存一下? (要抢 R_i)

$A := B \text{ op } C$

$RVALUE[R_i] = \{M_1, M_2 \dots M_n\}$.

$M_1: \begin{cases} A := B \text{ op } C, A \neq M_1 \\ M_1 := B \text{ op } M_1, B \notin RVALUE[R_i] \end{cases}$

若: $M_1 \notin AVALUE[M_1]$,

生成 $ST\ R_i, M_1$.

若: $\begin{cases} A := M_1 \text{ op } C \\ A := B \text{ op } M_1, B \in RVALUE[R_i] \end{cases}$

令 $AVALUE[M_1] = \{M, R_i\}$.

否则 令 $AVALUE[M_1] = \{M\}$.

删了 M_1 .

这是人类能读懂的?

要不要为 R_i 中的变量生成存数指令?

对 $RVALUE[R_i]$ 中每一变量 M , 如果 M 不是 A ,
 或者如果 M 是 A 又是 C , 但不是 B 并且 B 也不在
 $RVALUE[R_i]$ 中, 则

(1) 如果 $AVALUE[M]$ 不包含 M , 则生成目标代
 码 $ST\ R_i, M$

(2) 如果 M 是 B , 或者 M 是 C 但同时 B 也在
 $RVALUE[R_i]$ 中, 则令 $AVALUE[M] = \{M, R_i\}$.
 否则令 $AVALUE[M] = \{M\}$

(3) 删除 $RVALUE[R_i]$ 中的 M

4 给出 R , 返回.

<e.g.> $T := A - B$

$U := A - C$ R_0, R_1 空闲. W active.

$V := T + U$

$W := V + U$

IR	Obj code.	RVALUE	AVALUE
$T := A - B$	$LD\ R_0, A$ $SUB\ R_0, B$	$R_0 = \{T\}$	$T = \{R_0\}$
$U := A - C$	$LD\ R_1, A$ $SUB\ R_1, C$	$R_0 = \{T\}$ $R_1 = \{A\}$	$T = \{R_0\}$ $A = \{R_1\}$
$V := T + U$	$ADD\ R_0, R_1$	$R_0 = \{V\}$ $R_1 = \{A\}$	$V = \{R_0\}$ $A = \{R_1\}$
$W := V + U$	$ADD\ R_0, R_1$	$R_0 = \{W\}$ $R_1 = \{A\}$	$W = \{R_0\}$ $A = \{R_1\}$
	$ST\ R_0, W$		

<e.g> 11.1.

	RVALUE	VVALUE
$LD\ R_0, B$	$R_0 = \{T_1\}$	$T_1 = \{R_0\}$
$SUB\ B, C$		
$LD\ R_1, A$	$R_0 = \{T_1\}$	$T_1 = \{R_0\}$
$MUL\ R_1, R_0$	$R_1 = \{T_2\}$	$T_2 = \{R_1\}$
$LD\ R_0, D$	$R_0 = \{T_3\}$	$T_3 = \{R_0\}$
$ADD\ R_0, 1$	$R_1 = \{T_2\}$	$T_2 = \{R_1\}$
$ST\ R_1, T_2$		
$LD\ R_1, E$	$R_0 = \{T_3\}$	
$SUB\ R_1, F$	$R_1 = \{T_4\}$	
$MUL\ R_0, R_1$	$R_0 = \{T_5\}, R_1 = \{T_4\}$	
$LD\ R_1, T_2$		
$DIV\ R_1, R_0$	$R_0 = \{T_5\}, R_1 = \{T_2\}$	$W = \{R_1\}$
$ST\ R_1, W$		